8.1.1 첫 번째 Substrate 노드 실행하기

개요

이 튜토리얼에서는 Polkadot SDK를 사용하여 첫 번째 Substrate 노드를 실행하는 과정을 단계별로 안내합니다. 초보자도 쉽게 따라할 수 있도록 모든 과정을 상세히 설명하며, 각 단계에서 발생할 수 있는 문제와 해결 방법도 함께 제공합니다.

학습 목표

- Substrate 노드의 기본 구조 이해
- 개발 환경 설정 및 의존성 설치
- 노드 템플릿을 사용한 첫 번째 노드 실행
- 노드 상태 확인 및 기본 상호작용 방법 학습

사전 요구사항

- Rust 개발 환경 (1.70.0 이상)
- Git
- 기본적인 터미널 사용법
- 최소 8GB RAM, 50GB 디스크 공간

1단계: 개발 환경 준비

1.1 Rust 설치 확인

먼저 Rust가 올바르게 설치되어 있는지 확인합니다:

```
# Rust 버전 확인
rustc  --version
cargo  --version

# 필요한 경우 Rust 업데이트
rustup  update  stable
```

예상 출력:

```
rustc 1.75.0 (82e1608df 2023-12-21)
cargo 1.75.0 (1d8b05cdd 2023-11-20)
```

1.2 필수 도구 설치

Substrate 개발에 필요한 추가 도구들을 설치합니다:

```
# WebAssembly 타겟 추가
rustup  target  add  wasm32-unknown-unknown

# 필수 시스템 패키지 설치 (Ubuntu/Debian)
sudo  apt  update
sudo  apt  install  -y  git  clang  curl  libssl-dev  llvm  libudev-dev  make  p
```

```
# macOS의 경우
brew install openssl cmake llvm
```

1.3 환경 변수 설정

```
# ~/.bashrc 또는 ~/.zshrc에 추가
export PATH="$HOME/.cargo/bin:$PATH"
export RUST_BACKTRACE=1

# 변경사항 적용
source ~/.bashrc
```

2단계: Substrate 노드 템플릿 다운로드

2.1 노드 템플릿 클론

```
# 작업 디렉토리 생성
mkdir ~/substrate-projects
cd ~/substrate-projects

# Substrate 노드 템플릿 클론
git clone https://github.com/substrate-developer-hub/substrate-node-template
cd substrate-node-template
```

```
# 최신 안정 버전으로 체크아웃
git checkout latest
```

2.2 프로젝트 구조 확인

```
# 프로젝트 구조 확인
tree -L 2
```

예상 출력:

```
.
├── Cargo.toml
├── README.md
├── node/
├── pallets/
├── runtime/
└── scripts/
```

각 디렉토리의 역할:

- `node/` : 노드 실행 바이너리 코드
- `runtime/` : 블록체인 런타임 로직
- `pallets/` : 커스텀 팔렛 구현
- `scripts/` : 유틸리티 스크립트

3단계: 노드 빌드

3.1 의존성 확인

```
# Cargo.toml 파일 확인
cat Cargo.toml
```

3.2 노드 빌드 실행

```
# 릴리즈 모드로 빌드 (최적화됨, 시간이 오래 걸림)
cargo build --release

# 또는 개발 모드로 빌드 (빠름, 최적화 안됨)
cargo build
```

빌드 시간: 첫 번째 빌드는 20-40분 정도 소요될 수 있습니다.

빌드 성공 확인:

```
# 빌드된 바이너리 확인
ls -la target/release/node-template
```

3.3 빌드 문제 해결

일반적인 문제들:

1. **메모리 부족 오류:**

```
# 병렬 빌드 작업 수 제한
cargo build --release -j 2
```

2. **링커 오류:**

```
# 필요한 개발 도구 재설치
sudo apt install build-essential
```

3. **WebAssembly 타겟 누락:**

```
rustup target add wasm32-unknown-unknown
```

4단계: 노드 실행

4.1 개발 모드로 노드 시작

```
# 개발 체인으로 노드 실행
./target/release/node-template --dev

# 또는 임시 디렉토리 사용
./target/release/node-template --dev --tmp
```

성공적인 실행 로그:

```
2024-01-15 10:30:00 Substrate Node
```

```
2024-01-15 10:30:00 ✌️  version 4.0.0-dev-unknown
2024-01-15 10:30:00 ❤️  by Substrate DevHub <https://github.com/substrate-develop
2024-01-15 10:30:00 📋 Chain specification: Development
2024-01-15 10:30:00 🏷  Node name: flippant-boundary-1234
2024-01-15 10:30:00 👤 Role: AUTHORITY
2024-01-15 10:30:00 💾 Database: RocksDb at /tmp/substrate...
2024-01-15 10:30:00 🔨 Initializing Genesis block/state
2024-01-15 10:30:00 👴 Loading GRANDPA authority set from genesis
2024-01-15 10:30:00 🏷  Local node identity is: 12D3KooW...
2024-01-15 10:30:00 💻 Operating system: linux
2024-01-15 10:30:00 💻 CPU architecture: x86_64
2024-01-15 10:30:00 📦 Highest known block at #0
2024-01-15 10:30:00 〽️ Prometheus exporter started at 127.0.0.1:9615
2024-01-15 10:30:00 Running JSON-RPC HTTP server: addr=127.0.0.1:9933, allowed o
2024-01-15 10:30:00 Running JSON-RPC WS server: addr=127.0.0.1:9944, allowed ori
2024-01-15 10:30:06 🙌 Starting consensus session on top of parent 0x...
2024-01-15 10:30:06 🎁 Prepared block for proposing at 1 (0 ms) [hash: 0x...; pa
2024-01-15 10:30:06 🔖 Pre-sealed block for proposal at 1. Hash now 0x..., previ
2024-01-15 10:30:06 ✨ Imported #1 (0x...)
```

4.2 노드 실행 옵션

```
# 다양한 실행 옵션들
```

```
./target/release/node-template  --help


# 주요 옵션들:
# --dev: 개발 모드 (Alice 계정으로 자동 블록 생성)
# --tmp: 임시 데이터베이스 사용
# --port: P2P 포트 설정
# --rpc-port: RPC 포트 설정
# --ws-port: WebSocket 포트 설정
# --validator: 검증자 모드
```

4.3 커스텀 설정으로 실행

```
# 커스텀 포트로 실행
./target/release/node-template \
--dev \
--tmp \
--port 30334 \
--rpc-port  9934 \
--ws-port 9945
```

5단계: 노드 상태 확인 및 상호작용

5.1 RPC 엔드포인트 테스트

새 터미널을 열고 다음 명령어들을 실행합니다:

```
# 노드 정보 조회
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "system_name", "params":[]}' \
http://localhost:9933/

# 체인 정보 조회
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "system_chain", "params":[]}' \
http://localhost:9933/

# 최신 블록 번호 조회
curl -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "chain_getHeader", "params":[]}' \
http://localhost:9933/
```

예상 응답:

```
{"jsonrpc":"2.0","result":"Substrate Node","id":1}
{"jsonrpc":"2.0","result":"Development","id":1}
{"jsonrpc":"2.0","result":{"parentHash":"0x...","number":"0x5","stateRoot":"0x..
```

5.2 WebSocket 연결 테스트

```
# wscat 설치 (Node.js 필요)
npm install -g wscat

# WebSocket 연결 테스트
wscat -c ws://localhost:9944

# 연결 후 다음 메시지 전송:
{"id":1, "jsonrpc":"2.0", "method": "chain_subscribeNewHeads", "params":[]}
```

5.3 Polkadot.js Apps를 통한 GUI 접근

1. 브라우저에서 https://polkadot.js.org/apps/ 접속
2. 좌측 상단에서 “Local Node” 선택
3. “Switch” 버튼 클릭하여 로컬 노드에 연결
4. Explorer 탭에서 블록 생성 확인

6단계: 기본 트랜잭션 실행

6.1 계정 확인

개발 모드에서는 미리 정의된 계정들이 있습니다:

```
# Alice 계정 (기본 검증자)
# 주소: 5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY
# 시드: //Alice
```

```
# Bob 계정
# 주소: 5FHneW46xGXgs5mUiveU4sbTyGBzmstUspZC92UhjJM694ty
# 시드: //Bob
```

6.2 잔액 전송 테스트

Polkadot.js Apps에서:

1. “Accounts” 탭으로 이동
2. Alice 계정에서 “Send” 버튼 클릭
3. Bob 계정으로 1000 단위 전송
4. “Make Transfer” 버튼 클릭
5. “Explorer” 탭에서 트랜잭션 확인

6.3 CLI를 통한 트랜잭션

```
# subkey 도구 설치
cargo install --force subkey --git https://github.com/paritytech/substrate

# 새 계정 생성
subkey generate

# 계정 정보 확인
subkey inspect //Alice
```

7단계: 검증 및 문제 해결

7.1 노드 상태 검증 체크리스트

- [ ] 노드가 성공적으로 시작됨
- [ ] 블록이 정기적으로 생성됨 (6초마다)
- [ ] RPC 엔드포인트가 응답함
- [ ] WebSocket 연결이 작동함
- [ ] Polkadot.js Apps에서 연결 가능
- [ ] 기본 트랜잭션 실행 가능

7.2 일반적인 문제 해결

문제 1: 포트 충돌

```
# 사용 중인 포트 확인
netstat -tulpn | grep :9933

# 다른 포트로 실행
./target/release/node-template --dev --rpc-port 9934
```

문제 2: 데이터베이스 오류

```
# 데이터베이스 초기화
rm -rf /tmp/substrate*
./target/release/node-template --dev --tmp
```

문제 3: 메모리 부족

```
# 메모리 사용량 확인
free -h

# 스왑 메모리 추가 (필요시)
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

7.3 성능 모니터링

```
# 노드 메트릭 확인
curl http://localhost:9615/metrics

# 주요 메트릭들:
# - substrate_block_height: 현재 블록 높이
# - substrate_ready_transactions_number: 대기 중인 트랜잭션 수
# - substrate_peers: 연결된 피어 수
```

8단계: 다음 단계

8.1 추가 학습 자료

- Substrate 공식 문서
- Polkadot Wiki
- Substrate Recipes

8.2 다음 튜토리얼 준비

이제 다음 튜토리얼들을 진행할 준비가 되었습니다:

- 간단한 팔렛 개발하기
- 파라체인 템플릿으로 시작하기
- 커스텀 런타임 구성하기

요약

이 튜토리얼에서는 다음을 학습했습니다:

1. **환경 설정**: Rust 및 필수 도구 설치
2. **프로젝트 준비**: Substrate 노드 템플릿 다운로드
3. **빌드 과정**: 노드 컴파일 및 문제 해결
4. **노드 실행**: 개발 모드로 첫 번째 노드 시작
5. **상호작용**: RPC, WebSocket, GUI를 통한 노드 접근
6. **검증**: 노드 상태 확인 및 기본 트랜잭션 실행

다음 튜토리얼에서는 이 기본 노드를 바탕으로 커스텀 팔렛을 개발하는 방법을 학습하겠습니다.

참고 자료

- Substrate 노드 템플릿 GitHub
- Polkadot.js Apps

- Substrate 개발자 허브
- Rust 설치 가이드